EDGAR.jl: the SEC's filings and their numbers

What American companies file with the Securities and Exchange Commission, read into Julia: their filing histories, every number they have tagged in XBRL, the text of their filings, item by item.

In Pollis	The *US SEC EDGAR* panel, in Explore > Access Data Libraries: submissions, company facts, company concept, frames and full-text search, with the contact the SEC requires set once with *Set contact*.
Missing in Julia	a client for EDGAR's REST APIs that sends the User-Agent the SEC asks for, and readers that turn a filing into its items and its statements. Python has edgartools; Julia had ScrapeSEC.jl, which downloads the filing documents in bulk but does not read them.
Package	`EDGAR.jl` 0.8.0, from GitHub: `Pkg.add(url = "https://github.com/Trumpingtons/EDGAR.jl")`.
Tests	an offline suite, on saved filings and responses, that sends nothing to the SEC; and comparisons with edgartools on live filings: the items of eleven 10-Ks, GE's 10-K and nine 20-Fs to the character, and the face statements of 50 filers of unusual kinds, 386 of 389 values.

Why a package

EDGAR, the SEC's Electronic Data Gathering, Analysis and Retrieval system, holds the filings of the companies, funds and insiders it regulates, searchable by their text from 2001 on: annual reports (10-K), quarterly reports (10-Q), current reports (8-K), the annual reports of foreign issuers (20-F, 40-F), and many more. Since 2009 the financial statements come tagged in XBRL, and the SEC serves those tags as JSON on `data.sec.gov`, free and with no key (SEC, EDGAR APIs). Two things stand between those APIs and an analysis. Every request must say who sends it, or the SEC refuses it; and the numbers come as the filers tagged them, a quarter and a year mixed together, each period repeated as a comparative in the filings after it, revenue under one tag until 2017 and another since. EDGAR.jl handles the first and gives the tools for the second; for the filing itself, the document, it reads the items (*Item 1A. Risk Factors*, *Item 7. Management's Discussion*) and the statements, with the algorithms of edgartools (Gunning, edgartools) translated line for line.

Where it is used

- **Accounting and finance research.** A firm's reported numbers over time, or one number across every filer for one period, without a commercial database.
- **Text as data.** The risk factors or the management's discussion of many filings, for text analysis.
- **Event studies.** When each filing was accepted, to the second: the moment the market could read it.

The contact

The SEC asks every program that reads EDGAR to send a User-Agent that names the person or organisation and gives an email address, and to stay under ten requests a second (SEC, Accessing EDGAR Data); a request without one gets HTTP 403. EDGAR.jl takes the contact, on every request, from `set_user_agent("Name email")` if it was called in the session, and otherwise from the environment variable `SEC_USER_AGENT`; with neither, it stops with an error before it contacts the SEC.

In Pollis the contact is set once, with *Set contact* on the Powered-by line of the US SEC EDGAR panel. Pollis keeps it in VS Code's Secret Storage, encrypted by the operating system, and puts it in the environment of each Julia REPL and terminal it starts, as `SEC_USER_AGENT`, without writing it to a file or into the code. The panel's examples begin with `using EDGAR` and nothing else: the contact is already there.

What it reads

A CIK, the SEC's Central Index Key, identifies a filer; every function takes it as a number or a string, with or without its leading zeros (320193 or `"0000320193"` for Apple).

Function	Returns
`cik("AAPL"; by = :ticker)`	the CIKs of the SEC's ticker list that match a ticker or a name
`profile(cik)`	the filer: name, SIC industry, fiscal year end, tickers, exchanges, former names
`filings_by_cik(cik; forms, startdate, enddate)`	one row per filing, newest first: form, period, date filed, acceptance time in UTC, accession number, primary document, XBRL flags
`full_text_search(query; forms, …)`	the filings whose text contains a phrase, ranked by relevance
`company_facts(cik)`	every XBRL fact the filer has reported, by taxonomy and concept
`company_concept(cik, taxonomy, tag)`	one concept over time, by unit
`xbrl_frames(taxonomy, tag, unit, period)`	one concept for one calendar period, across every filer
`fetch_filing(cik, accession)`	a filing's document, in memory, to read
`sections(f)`, `TenK`, `TwentyF`	a filing's items, in order
`facts(f; classify = true)`	a filing's facts, each with its statement and with the signs as reported

Search and filing lists come 100 rows to a page, as EDGAR's full-text search gives them, with no total count: pass `from = 100, 200, …` for the next pages. The rows are vectors of named tuples, so any Tables.jl reader takes them. Beyond the SEC, the package reads the filing systems of other countries (the ESEF reports of the European Union, Companies House, EDINET, DART and others); this vignette is about the SEC.

A worked example

Apple's net income, a fact a year

```
using EDGAR                                                  # the contact comes from Set contact
p = profile(320193)                                         # Apple Inc., fiscal year ending 0927
c = company_concept(320193, "us-gaap", "NetIncomeLoss")
annual = filter(o -> haskey(o, :frame) && !occursin('Q', o.frame), c.units.USD)
```

`c.units.USD` has every net income Apple has tagged: quarters and years, each repeated as a comparative in the two filings after it. The SEC marks with a `frame` (`CY2023`, `CY2023Q1`) the one fact that best fits each calendar period, so the filter keeps one fact a year. Apple's year ends in September, so its fiscal 2023 is framed as `CY2023`, the calendar year it overlaps most. Across filers, frames do the same (`xbrl_frames("us-gaap", "Assets", "USD", "CY2022Q4I")`): a trailing `I` asks for an instant, a balance at the end of the period, and no `I` for a flow over it.

When a filing was accepted

The submissions API, `data.sec.gov/submissions`, gives each filing's `acceptanceDateTime` as UTC, but it is New York time plus twice New York's offset: four hours late in summer, five in winter. EDGAR's own filing index pages and Atom feeds give the right time. `filings_by_cik` takes the offset back out, with New York's daylight-saving rules:

Accepted, New York	Submissions API	EDGAR.jl
19 Oct 2026, 18:32:55 EDT	2026-10-20T02:32:55.000Z	2026-10-19T22:32:55.000Z
29 Jan 2026, 18:34:19 EST	2026-01-30T04:34:19.000Z	2026-01-29T23:34:19.000Z

The rows are two of Apple's filings, out of nine checked against their index pages. For an event study of filings, the difference is the difference between after the close and the next morning.

What the tests show

Offline

`test/runtests.jl` runs on saved filings and responses in `test/data` and sends no request to the SEC: the CIK forms, the acceptance times above, the readers of items and tables, the XBRL parsers, the classification of statements, the cache. It needs no contact, so it runs in continuous integration.

Against edgartools

On live filings, EDGAR.jl was compared with edgartools 5.39 (Python):

Compared	Result
Items of eleven 10-Ks of large companies, Apple, Johnson & Johnson, JPMorgan, Coca-Cola, Exxon, Boeing, Walmart, Chevron, Pfizer and 3M among them	the same text, to the character
GE's 10-K, whose items are found through its cross-reference index	23 of 23 items, to the character
Nine 20-Fs (SAP, Novartis, TSMC, AstraZeneca, Toyota, Sony, Honda, Novo Nordisk, Lloyds), filed through three filing agents or by the company itself	every item, to the character
Face statements of 50 filers chosen for unusual structures (partnerships, foreign issuers under other GAAPs, REITs, utilities, insurers, asset managers)	386 of 389 values; the other 3 a choice of the comparison, not of either package
Ten 10-K/A amendments, all small companies	42 of 42 values

The comparisons found faults on both sides. EDGAR.jl's were fixed: slips in the translation of edgartools' item readers (a byte too many at the end of an item, an empty string that Python reads as false, the order of a Python dictionary), two names of income statements it did not know (*Statement of Earnings*, *Comprehensive Earnings*), and the negated labels: a filer can store a cash outflow as a positive number and show it with a minus sign, through the label of its presentation. 71 of the 72 filers checked do this, on a median of 23 lines; `facts(f; classify = true)` now gives the numbers as the statements show them, and `classify = false` as stored, as `company_facts` does.

Still open

- **Some layouts.** Neither package finds the items of Tesla's 10-K; in a 20-F of 53 MB (Santander) EDGAR.jl finds 3 items, and both packages take minutes to read it.
- **The rate.** EDGAR.jl does not hold its requests under the SEC's ten a second; a loop over many filers has to pace itself. Responses are cached, for the session by default.
- **The panel.** The US SEC EDGAR panel shows the five data APIs; reading a filing into its items and statements is in the package but not yet in the panel.

References

Gunning, D. edgartools: a Python library for SEC EDGAR filings (MIT). `https://github.com/dgunning/edgartools`

U.S. Securities and Exchange Commission. Accessing EDGAR Data: fair access and the User-Agent. `https://www.sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data`

U.S. Securities and Exchange Commission. EDGAR Application Programming Interfaces. `https://www.sec.gov/search-filings/edgar-application-programming-interfaces`

Software: HTTP.jl, JSON3.jl, EzXML.jl, OrderedCollections.jl and ZipArchives.jl (MIT); concept mappings vendored from edgartools (MIT).

Package written and tested with Claude Code (Anthropic) in Pollis.

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 28 October 2026

contact@pollis.org